

Histórico das Linguagens e Introdução à Orientação a Objetos

Programação Orientada a Objetos - 2026/1

Prof. Dr. Valter Vieira de Camargo
Departamento de Computação - UFSCar

Um pouco de história

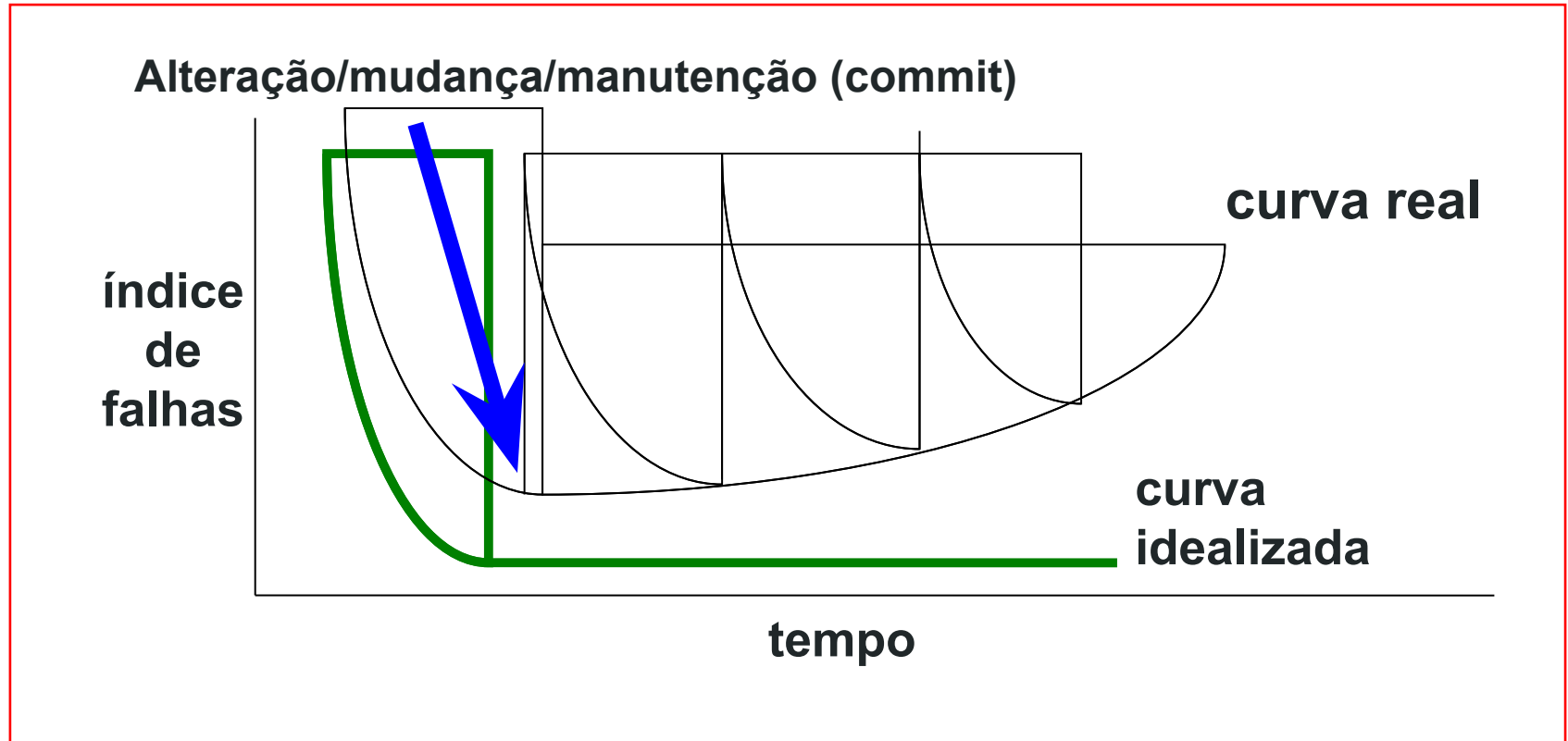
O que é “software” ?

- O software é composto por dois elementos principais:
 - Instruções
 - quando executadas produzem a função e o desempenho desejados,
 - conhecido por vários nomes: métodos, procedimentos, funções, operações
 - manipulam os “dados”
 - Dados
 - onde as informações ficam armazenadas

Características do Software

- Seu desenvolvimento envolve muita criatividade, não é construído como um produto físico.
- Não se desgasta, mas se deteriora com o tempo.
 - Após muitas manutenções, tende a apresentar comportamentos indesejáveis
 - Baixa manutenibilidade
 - Erosão arquitetural
- Parcialmente reutilizável, mas dificilmente por completo.

Características do Software



CURVA DE FALHAS DO SOFTWARE

Evolução do Software

1965 – 1975

- Sistemas de tempo real.
- 1ª geração de SGBD's.
- Produto de software - *software houses*.
- Bibliotecas de Software.
- Cresce o número de sistemas baseado em computador.
- Manutenção quase impossível.
- 1967 → Começa-se a falar de **Orientação a Objetos** na linguagem Simula
- 1970 → SmallTalk → primeira linguagem puramente orientada a objetos

..... **CRISE DE SOFTWARE**

Evolução do Software

1975 – 2000

- Sistemas distribuídos.
- Redes locais e globais.
- Uso generalizado de microprocessadores
- 1980 → C++ coloca OO no mundo de desenvolvimento de sistemas
- 1990
 - → Java populariza OO no desenvolvimento comercial
 - → Tim Berners-Lee criou o conceito de páginas interligadas por hiperlinks, dando origem à web.
- 1993 → O primeiro navegador (browser) foi lançado, chamava-se Mosaic
- 1995
 - Uso comercial da Internet (Yahoo, eBay, Amazon, ...)
 - Primeiras conexões internet domésticas no Brasil (via discador)

Evolução do Software

2000... Quarta era do software

- OO continua forte..
- Sistemas especialistas e software de inteligência artificial
- Software de rede neural artificial.
- Computação Paralela.
- Grids, clouds, etc.

Evolução do Software

2000 - 2010

- **Domínio da Programação Orientada a Objetos (OO):** Java, C# e Ruby ganharam popularidade.
- **Expansão da Web:** PHP, JavaScript (com AJAX), Ruby on Rails e frameworks web começaram a se destacar.
- **Concorrência e multiprocessamento:** Interesse crescente em linguagens como Erlang.
- **Linguagens de scripting:** Python começou a ganhar mais espaço.
- **Influência do Open Source:** O crescimento do Linux e projetos abertos impactaram a adoção de linguagens.

Evolução do Software

2010 - 2020

- **Crescente adoção do paradigma funcional:** Scala, Clojure, Kotlin e Elixir ganharam relevância.
- **Ascensão do JavaScript no back-end:** Node.js revolucionou o desenvolvimento web.
- **Programação reativa:** Surgimento de bibliotecas e frameworks como RxJS e React.
- **Desenvolvimento mobile:** Swift (substituindo Objective-C) e Kotlin (para Android).
- **Machine Learning e Data Science:** Python se tornou dominante, enquanto R também cresceu.

Evolução do Software

2020 ...

- **Domínio da Inteligência Artificial e Ciência de Dados:**
Python continua liderando, e novas linguagens como Julia surgem.
- **Linguagens específicas para segurança e desempenho:**
Zig e Carbon (iniciativa do Google).
- **Low-code e no-code:** Expansão de ferramentas que permitem desenvolvimento sem código.

IEEE Spectrum Ranking 2025



Importância do Software Atualmente

- **Infraestrutura crítica**

- Controle de energia elétrica, abastecimento de água e telecomunicações dependem de software.
- Um erro pode causar apagões em larga escala ou falhas em serviços essenciais.

- **Saúde e Medicina**

- Sistemas hospitalares gerenciam prontuários, exames e equipamentos médicos.
- Uma falha pode atrasar diagnósticos, administrar medicamentos errados ou falhar em cirurgias robóticas.

- **Bancos e Finanças**

- Transações financeiras, bolsa de valores e pagamentos online dependem de software.
- Falhas podem causar perdas bilionárias, fraudes e instabilidade econômica.

- **Transportes e Aviação**

- Carros autônomos, controle de tráfego aéreo e sistemas ferroviários são altamente automatizados.
- Um bug pode causar acidentes fatais e grandes atrasos.
- Sistemas de defesa militar, controle de fronteiras e cibersegurança dependem de software confiável.
- Falhas podem expor informações sigilosas ou comprometer a segurança nacional.

- **E-commerce e Logística**

- Compras online, estoques e entregas globais são gerenciados por software.
- Falhas podem levar a perdas financeiras, atraso em produtos e caos no comércio.

- **Redes Sociais e Comunicação**

- Aplicativos como WhatsApp, Instagram e Zoom mantêm bilhões de pessoas conectadas.
- Um erro pode derrubar comunicações globais ou expor dados de usuários

Orientação a Objetos

Orientação a Objetos

- Definição:
 - Uma maneira de trazer para dentro de um programa, o que realmente ocorre no mundo real;
 - O componente principal de decomposição é o “objeto” ou “classe”. Que combina “estrutura” e “comportamento” em uma única entidade - **a Classe**.
- Orientação a Objetos é uma forma de programação, assim como a procedimental, orientada à eventos, etc.
- É uma forma de organizar o código

Outras definições

Alan Kay (criador do Smalltalk e um dos pais da OO)

- *"A programação orientada a objetos é mais sobre mensagens do que sobre classes".*
- Alan Kay enfatizava que OO não é apenas sobre classes e herança, mas sobre a **comunicação entre objetos** por meio de troca de mensagens.

Grady Booch (autor de "Object-Oriented Analysis and Design with Applications")

- *"A programação orientada a objetos é um método de desenvolvimento de software no qual os componentes do sistema são **objetos interagindo entre si**".*
- Ele destacou que OO é uma forma de organizar software baseada em objetos que encapsulam dados e comportamentos.

Bjarne Stroustrup (criador do C++)

- *"A programação orientada a objetos é simplesmente programação com abstração de dados".*
- Ele focou na ideia de que OO é uma extensão natural do conceito de **abstração**, permitindo modelar sistemas complexos.

Bertrand Meyer (criador do Eiffel e do conceito de "Design by Contract")

- *"OO é a técnica que usa objetos como unidades básicas de modularidade, combinando estrutura de dados e comportamento em entidades coesas".*
- Meyer reforçou a importância da **modularidade e coesão** como princípios fundamentais da OO.

Orientação a Objetos

- Principais Conceitos da OO
 - Abstração
 - Classes/Objetos
 - Comunicação por Mensagens
 - Herança (Generalização/Especialização)
 - Composição (associações entre objetos)
 - Polimorfismo **
 - Encapsulamento**
 - visibilidade entre elementos

Conceitos OO

Abstração

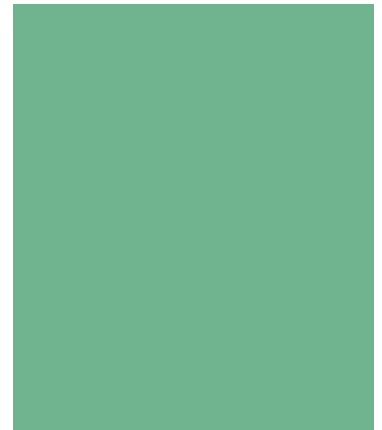
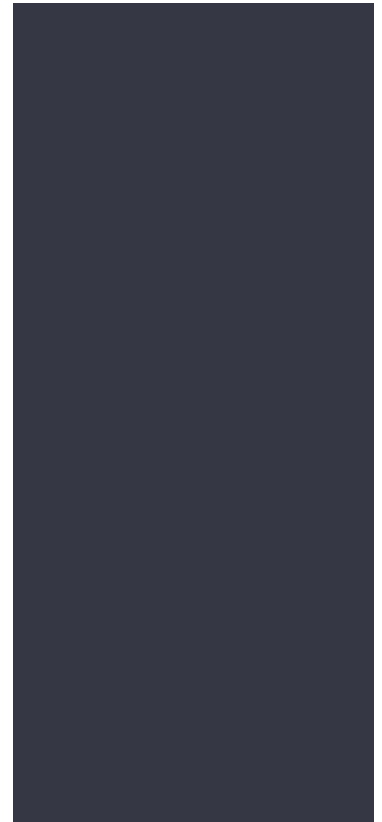
- Capacidade de se concentrar em detalhes de um problema, ignorando outros não relevantes para o entendimento/resolução desse;
- Uma “boa” abstração revela apenas os detalhes importantes de interesse para um determinado observador.
- Conceito Caixa Preta



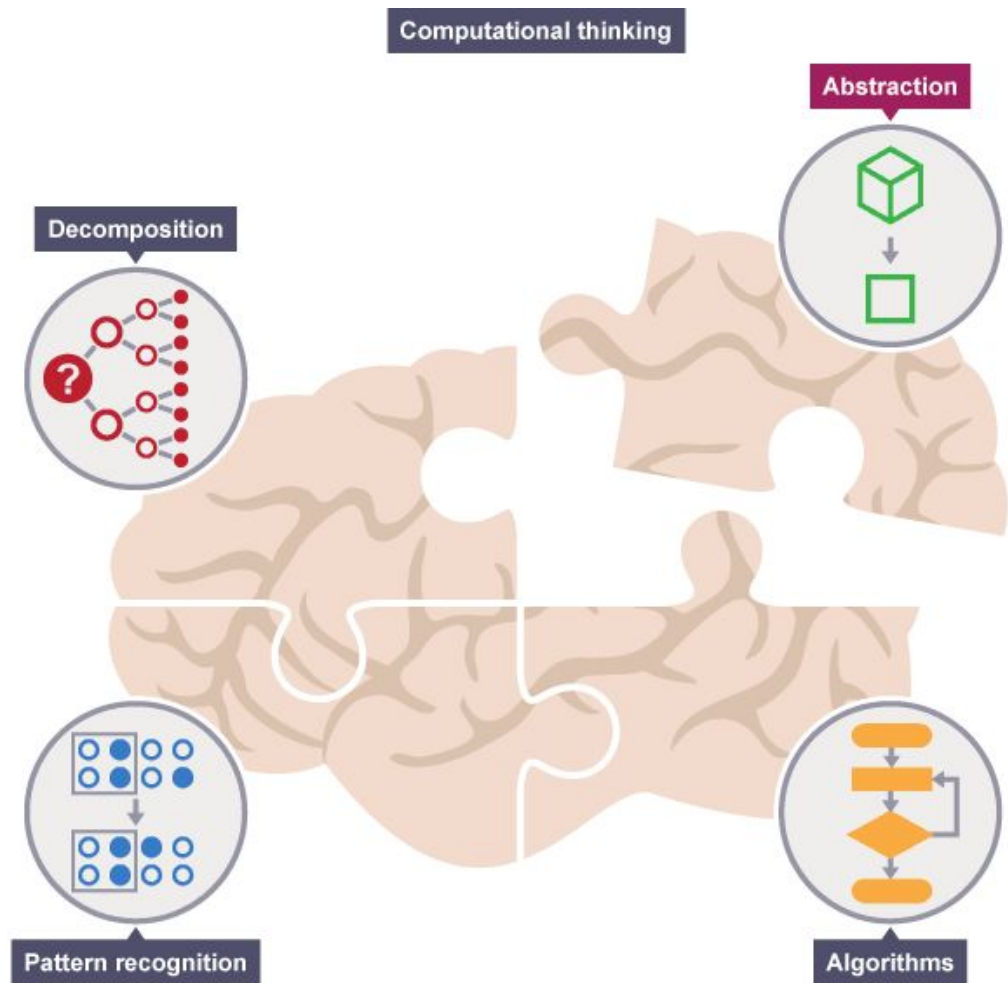
ABSTRAÇÃO

Abstraction is the process of filtering out – ignoring - the characteristics of patterns that we don't need in order to concentrate on those that we need. It is also the filtering out of specific details. From this we create a representation (idea) of what we are trying to solve.

*Computer science, being primarily concerned with developing interaction patterns, has information hiding as its abstraction objective.
(Colburn T., Shute G. 2007)*



THE FOUR CORNERSTONES OF COMPUTER SCIENCE

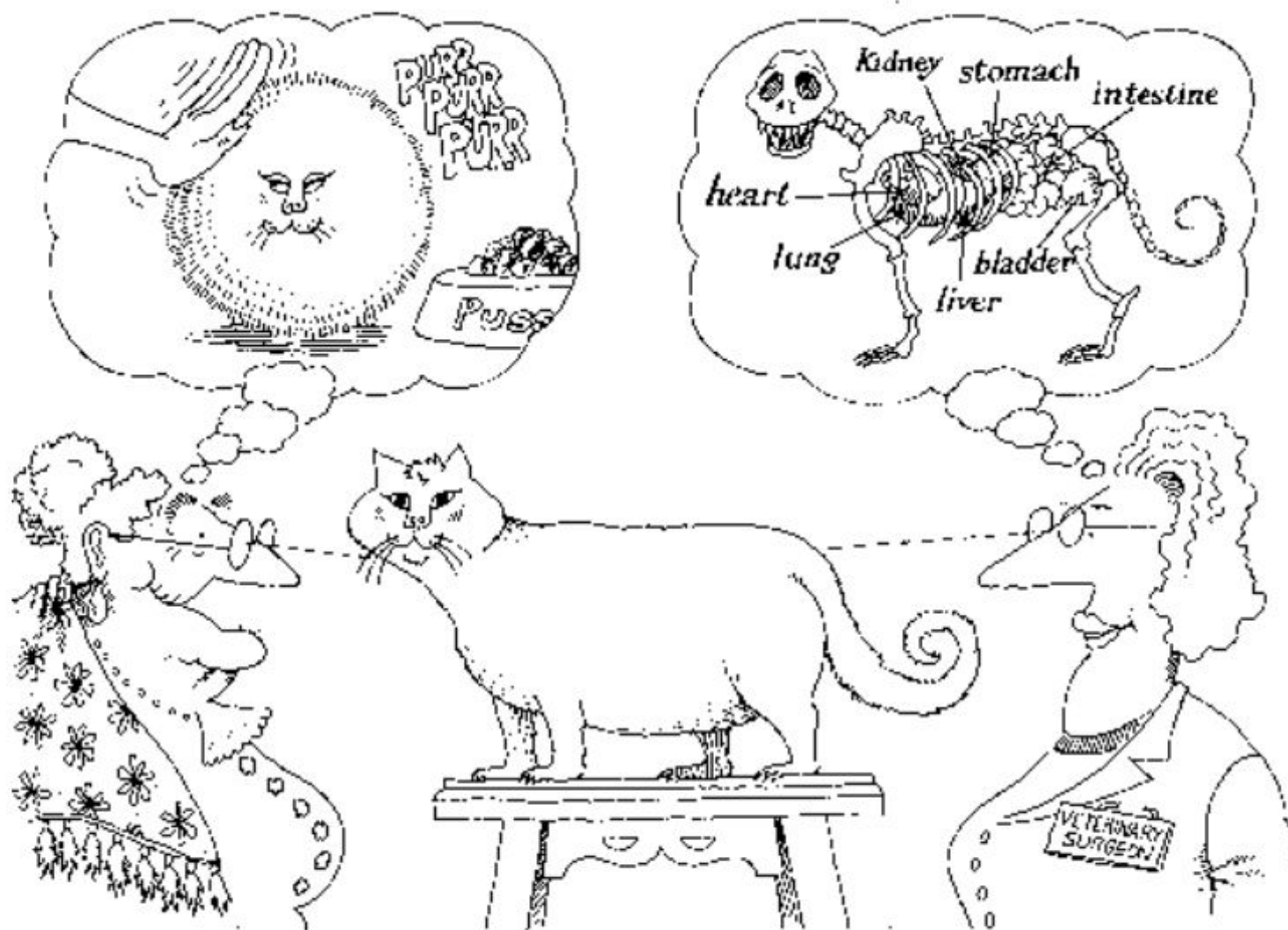


Conceitos OO

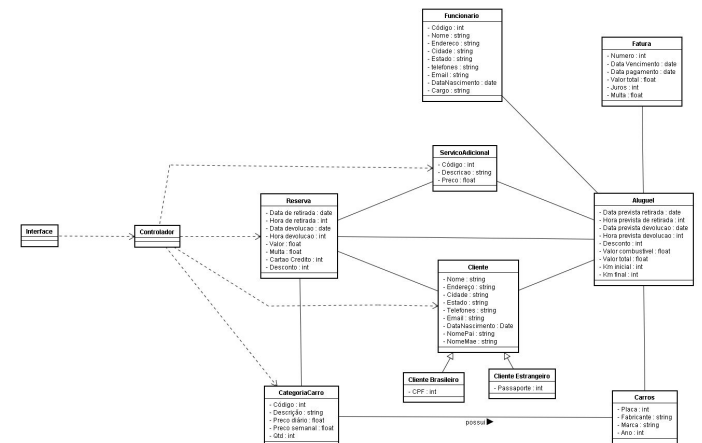
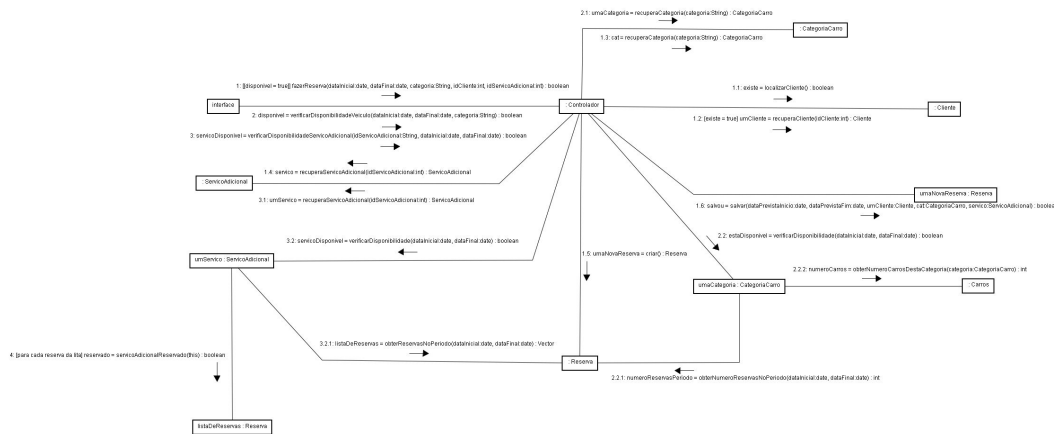
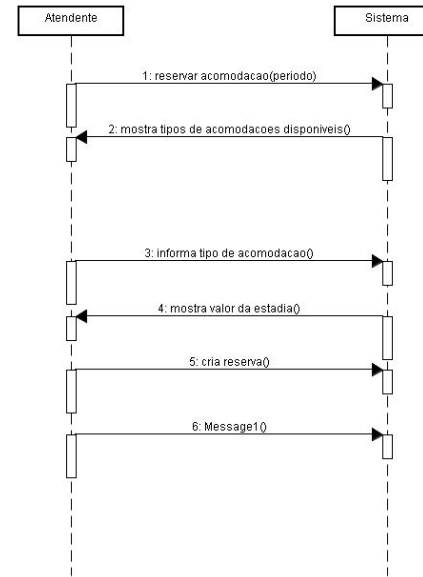
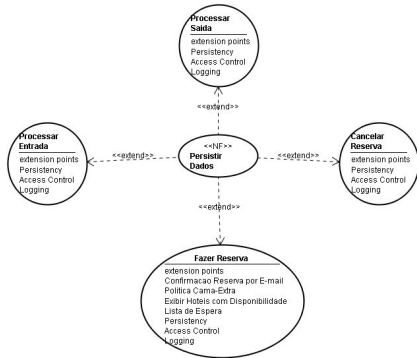
Abstração

Uma classe é uma
abstração do
mundo real

- Como decompor um sistema em classes ?
 - O que deve se tornar uma classe ?
 - Que nome deve ter essa minha classe ?
 - Ela tem um nome adequado ?
 - Normalmente, classes são substantivos → Cliente, Produto, Nota Fiscal, etc.
 - Não é normal ter classes com nomes que são Verbos → Comprar, FinalizarPedido, GerarRelatorio, Armazenar....
- Uma boa pergunta que pode te ajudar a decidir se está criando uma boa classe é a seguinte:
 - Quais métodos devo colocar nessa classe ?
 - Os métodos que estou colocando fazem sentido nela ?
- Modelar um sistema somente com abstrações do mundo real pode representar o presente, mas não o futuro 😞
- As abstrações usadas durante o projeto são a chave para um projeto fácil de manter e evoluir



Abstrações na UML (Unified Modeling Language)



Conceitos da OO

Classes e Objeto

- Como capturar dentro de uma linguagem de programação um objeto do mundo real?
 - Um objeto é uma ocorrência específica de uma **classe** e é similar a uma entidade/tabela no modelo relacional;
 - Por exemplo: “Valter” é um objeto da classe “Pessoa”. Uma classe define os dados e o comportamento de seus objetos.
 - Valter é uma instância da classe “Pessoa”
- ***** Cada classe criada é uma abstração *****
 - *Sempre pense → quais são as abstrações que você está usando em seu sistema ?*

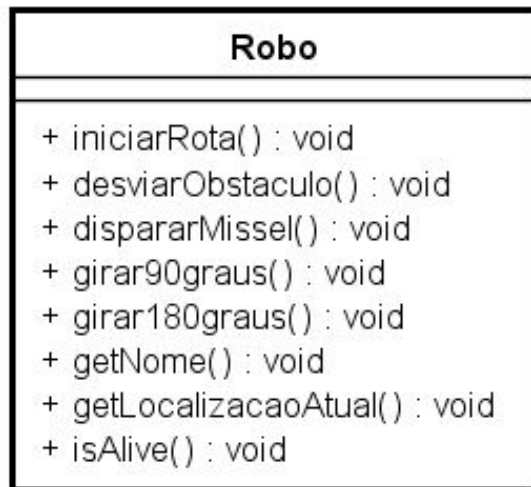
Conceitos da OO

Classes e Objeto

- Importante observar que uma classe encapsula:
 - **dados** → valores de atributos/variáveis
 - **serviços** → métodos/operações/funções
- Ao olhar para os métodos de uma classe sabemos quais são os **serviços** que ela oferece
 - os métodos são os serviços que podemos pedir/requisitar para ela realizar
 - **O conjunto de métodos é o que a classe sabe fazer**

Conceitos da OO

Classes e Objeto



```
.... {Linguagem genérica ...}  
  
Robo meuRobo = new Robo()  
  
meuRobo.iniciarRota()  
...  
if (tem obstaculo) then  
    meuRobo.desviarObstaculo();  
...  
meuRobo.dispararMissel()  
...  
meuRobo.girar90graus  
...  
  
Print ("O nome do meu robô é : " + meuRobo.getNome())  
...
```

Conceitos da OO

Mensagens entre objetos

- Como fazer os objetos conversarem?
 - Um mensagem é um sinal enviado de um objeto a outro requisitando um **serviço** por meio da execução de uma operação/método
 - Essa operação é executada dentro da classe com base **nos dados de seus alcance na hierarquia de classes** e uma mensagem resultante é retornada ao solicitante (callback)
 - Nada mais é do que uma chamada de método ...

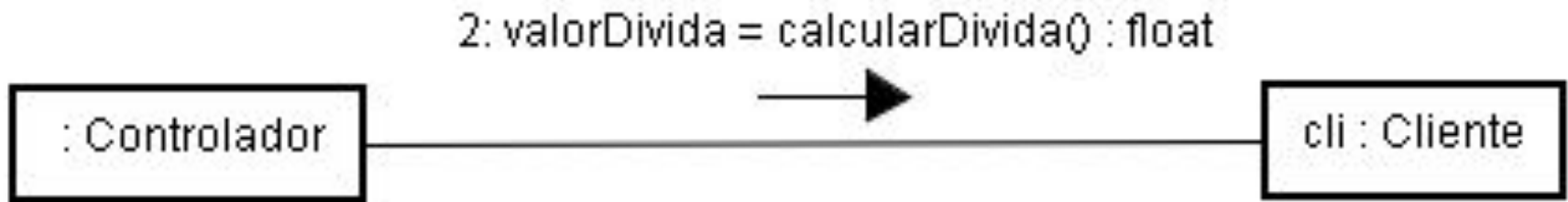
Conceitos da OO

Mensagens

- Como fazer os objetos conversarem?
 - Para que uma msg seja enviada deve existir um canal de comunicação entre os objetos. Isso é concretizado na forma de **relacionamentos** entre as classes.
 - Os relacionamentos são criados por meio da declaração de variáveis de uma classe dentro da outra ...

Conceitos da OO

Mensagens

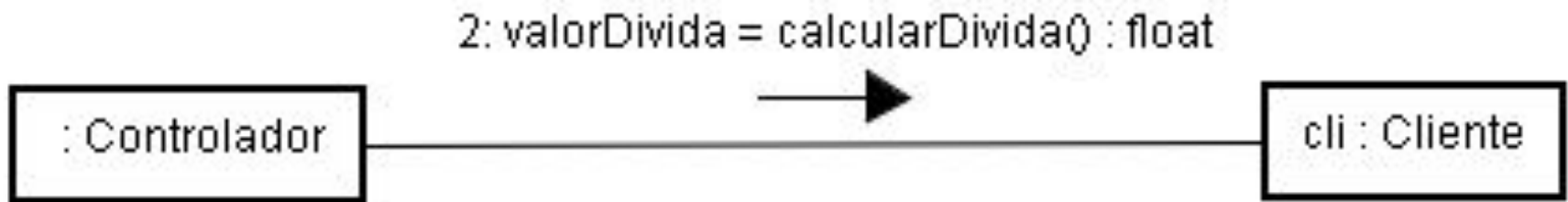


```
class Controlador {  
  
    public m() {  
        cli = new Cliente(...);  
        ....  
        float valorDivida = cli.calcularDivida();  
        ....  
    }  
}
```

```
class Cliente {  
  
    public float calcularDivida()  
    {  
        ...  
        return valorDivida;  
    }  
}
```

Conceitos da OO

Mensagens



```
class Controlador {
```

```
    public m() {
```

```
        cli = new Cliente(...);
```

```
        ....
```

```
        float valorDivida = cli.calcularDivida();
```

```
        ....
```

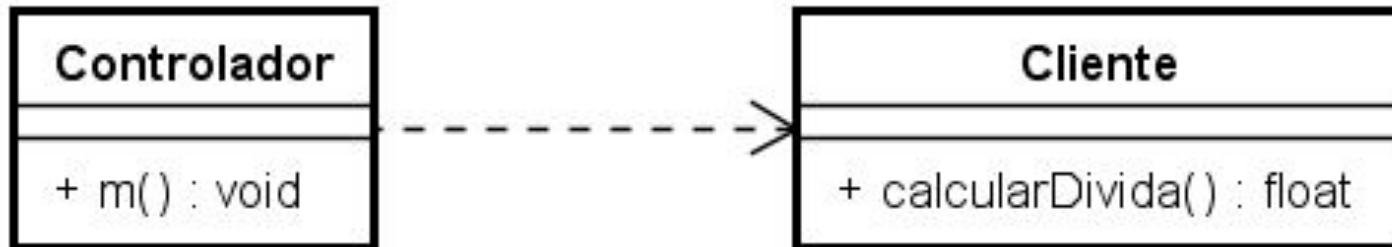
```
class Cliente {
```

```
    public float calcularDivida()
```

```
    {
```

```
        ...
```

```
        return valorDivida;
```



Encapsulamento

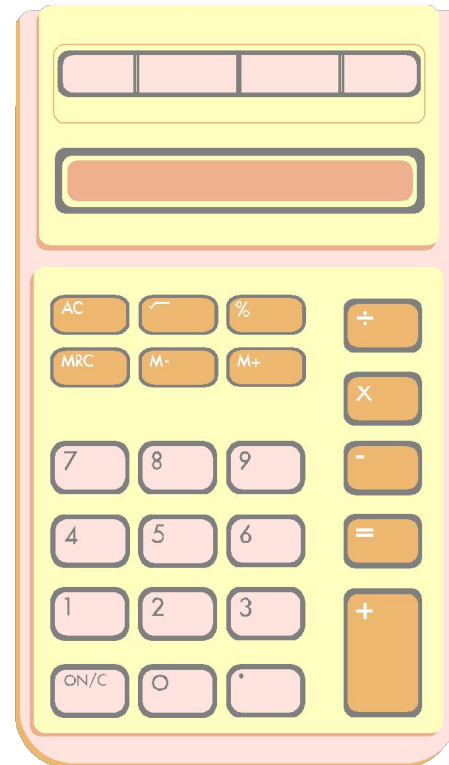
- Esse conceito está relacionado à proteger os dados da classe

Encapsulamento

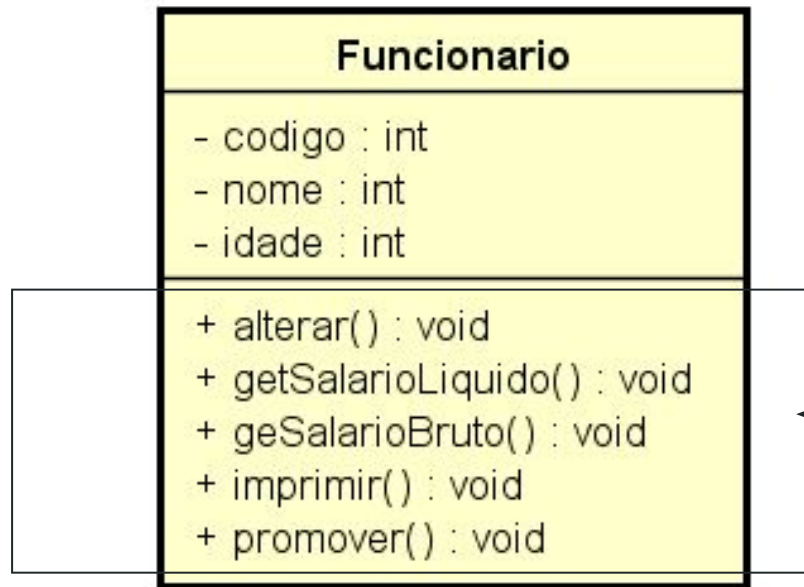
- Cada componente do programa deve conter uma única funcionalidade
- A interface do componente deve expor o mínimo possível sobre o funcionamento interno do componente
- Usuários de uma biblioteca necessitam saber apenas sua interface (assinatura dos métodos) para utilizá-la. Alterações no algoritmo não afetam os usuários que a utilizam.
- Combina-se atributos e serviços que agem sobre esses atributos

Encapsulamento – ex.

- Agrupa seus registradores internos e disponibiliza ao usuário apenas as funções necessárias.
- Não se tem acesso ao seu Interior.
- é uma caixa preta



Encapsulamento – ex.



powered by Astah

Para utilizar a classe deve-se conhecer apenas a interface pública que ela fornece.

+ público
- privado
protegido

FIM